

6 BRANCHING AND LOOPING

In this chapter we will learn about types of statements supported by Python statement set. Table 6.1 summarizes the types of statements supported in Python and you have already seen top two i.e. assignment and print in Chapter 3. Table 6.1 also highlights the respective Section of this book, where we have explained the details about individual statement types. In your programs, these units of code can perform actions, repeat tasks, to break out or continue with loop and so on.

This chapter deals with the entries listed in Table 6.1 and explains their working process based on data type need to work with or depends on the situation. Statements lower in Table 6.1 i.e., **def**, **class**, **try/catch**, and **import** that have to do with larger program units—functions, classes, modules, and exceptions—lead to larger programming ideas, so we have not included here. But, **def** statements are discussed separately in Chapter 8 and rest of them included in the Part 2 of this book. More focused statements (like **del**, which deletes various components) are covered in Chapter 7.

Table 6.1 Python Statement Types

Statement	Role	Example
assignment	Creating Reference	Discussed in Chapter 3, 4
print calls	Printing Object	print(var)
if/elif/else	Selection	Discussed in Section 6.1
for/else	Iteration	Discussed in Section 6.3 and 6.5
while/else	Iteration	Discussed in Section 6.3 and 6.5
pass	Empty placeholder	Discussed in Section 6.6
break	Exit from loop	Discussed in Section 6.4
continue	Loop continue	Discussed in Section 6.4
match	Branching with multiple case	Discussed in Section 6.2
del	To remove individual items from collections	Discussed in Chapter 7
def	Defining a function	Discussed in Chapter 8
class	To define a user-defined type	Not included in this book
import	To import an external module	Not included in this book
try/catch	To handle exception	Not included in this book

Everything you have seen so far has consisted of sequential execution, in which statements are always performed one after the next, in exactly the order specified. But the world is often more complicated

than that. Frequently, a program needs to skip over some statements, execute a series of statements repetitively, or choose between alternate sets of statements to execute. That is where control structures come in. A control structure directs the order of execution of the statements in a program.

6.1 Selection Statement

In the real world, we commonly evaluate information around us and then choose one course of action or another based on what we observe. If the weather is nice, then I'll mow the lawn. (It's implied that if the weather isn't nice, then I won't mow the lawn.)

In a Python program, we have used **if** statement to perform decision-making. It allows for conditional execution of a statement or group of statements based on the value of an expression. This section highlighted the working of **if** statement in following ways:

- First, you'll get a quick overview of the if statement in its simplest form.
- Next, using the if statement as a model, you'll see why control structures require some mechanism for grouping statements together into compound statements or blocks. You'll learn how this is done in Python.
- Lastly, you'll tie it all together and learn how to write complex decision-making code.

6.1.1 The if Statement

The simplest form of **if** statement is,

```
>>> if expr:
...     statement(s)
```

The above form can be broken down as follows:

- `expr` is an expression evaluated in a Boolean context, either as `True` or `False`, as we have discussed in Section 4.2 and 4.3 of Chapter 4.
- `Statement(s)` refers one or many valid Python statement, starts after indentation.
- If `expr` is `True` (evaluates to a value that is “truthy”), then `statement(s)` executed. If `expr` is `False`, then `statement(s)` is skipped over and not executed.
- The colon (`:`) following `expr` is required, following this body of the **if** statement will start. Some programming languages require `expr` to be enclosed in parentheses, but Python does not.

Next, we have listed some examples to understand the working process of **if** statement.

```
>>> if 5 > 4:
...     print('Yes 5 > 4')
...
Yes 5 > 4
>>> if 5 < 4:
...     print('Yes 5 < 4')
...

```

The condition of the **if** statement `5 > 4`, evaluates to `True` and the `print` statement inside if gets executed. The outcome of the above execution is displaying a message `'Yes 5 > 4'`. But the second if condition, `5 < 4`, evaluates to `False` and that's why we did not received any output. In the next example, two conditions are combined with logical operators in `expr` of the **if** statement.

```
>>> if True or True:
...     print('Truthy Condition')
...
Truthy Condition
>>> if False or False:
...     print('Falsy Condition')
...
```

For this example, we have used Boolean values `True` and `False` with logical **OR** operator. The first if expression evaluates to `True` and the print statement 'Truthy Condition' gets executed. But the logical **OR** returns `False`, when both the operands evaluate to `False` and that is the reason, body of second if statement is not executed means skipped or stepped over by the interpreter. Now let's see a bit complex expr and mix comparison and logical operator to form an expression within an if statement:

```
>>> if 4 == 4 and 5 > 3:
...     print('Truthy Condition')
...
Truthy Condition
```

In the above example, two conditions `4 == 4` and `5 > 3` are combined using logical **and** operator. Both the conditions are evaluating to `True` and the body of if, gets executed and displayed the message. There is no such limit, how many such conditions we can combine using logical operators. Let's solve some problem to understand the working of if statement. All these are executed through Python script file (.py). See Chapter 2, how to create and execute Python script files using IDLE.

Problem 6.1: Write a program to find the largest of two integers using if statement only and numbers must be taken as input from user.

```
1 #Step-1: Taking Input
2 num1 = int(input('Enter number:'))
3 num2 = int(input('Enter number:'))
4 #Step-2: Display both numbers
5 print(f'Numbers are : num1 = {num1}, num2 = {num2}')
6
7 #Step-3: Condition for num1 is largest or step-4
8 if num1 > num2:
9     print(f'{num1} is largest')
10 #Step-4: Condition for num2 is largest
11 if num2 > num1:
12     print(f'{num2} is largest')
```

```
Enter number:23
Enter number:12
Numbers are : num1 = 23, num2 = 12
23 is largest
```

Two numbers 23 and 12 is taken as input and the expr in first if, **num1 > num2** line 8, evaluates to

True and displayed '**23 is largest**'. The expr, at line 8, written to test num1 (23) is larger than num2 (12), and the expr, evaluates to True. The second expr **num2 > num1**, at line 11, written for num2 is larger than num1, becomes False for this pair of input. Till this discussion we have found that body of if statement only executed if the expression of if evaluates to True. All the above examples have two if statements, one written for Truthy condition and other one for Falsy condition. One can change the pair of input, as shown below, and test the expr at line 11. So that **num2** become largest.

```
Enter number:5
Enter number:6
Numbers are : num1 = 5, num2 = 6
6 is largest
```

In the above program multiple if statements are written for same purpose (to check the largest) and the if statement, **num1 > num2** at line 8, evaluates to True. From this point one can understand this, out of these two if statements, one will be True and other will be False and need to skip. But Python interpreter, evaluates both, because we have not written the code to do this. It can be achieved by introducing **else** (works for False condition) clause with **if** statement. Before introduction to the else clause let us understand the importance of indentation for if statement. Consider the coding sample illustrated next:

```
1 num1 = int(input('Enter number:'))
2 num2 = int(input('Enter number:'))
3 print(f'Numbers are : num1 = {num1}, num2 = {num2}')
4
5 if num1 == num2:
6     print('Start of If')
7     print('Both numbers are same')
8     print('End of If')
9 print('After If')
```

In the above code, if the condition **num1 == num2**, is True, then all the indented statements from line 6 to 8 get executed and then non-indented statement at line 9 will be executed. If the condition become False, then program control will skip all the indented lines from 6 to 8 and directly executed line 9. See the program output:

```
Enter number:4
Enter number:4
Numbers are : num1 = 4, num2 = 4
Start of If
Both numbers are same
End of If
```

For the same input, the condition at line 5, evaluates to True and all the indented lines inside if, gets executed and then non-indented lines. Now execute the script again and give two different numbers as input, as shown below:

```
Enter number:4
Enter number:5
```

Numbers are: num1 = 4, num2 = 5

After If

For different numbers as input and the condition at line 5, evaluates to False and program control skipped all indented line inside if, and directly executes the non-intended lines after if. In both the scenarios we have understood that, either way, execution proceeds with line 9 afterward. Did you notice that there is no way to know the end of if block? Rather, put a print statement at the last intended line inside if.

Note: In the Python documentation, a group of statements defined by indentation is often referred to as a suite. This book uses the terms block instead of suite.

6.1.2 The else Clause

We have learnt, how to use an if statement to conditionally execute a single statement or a block of statements. It's time to find out what **else** can do. Sometimes, you want to evaluate a condition and take one path of execution, if the condition with **if** statement it is true, and specify an alternative path if it is not. This is accomplished with an **else** clause after an **if**. Let see the general syntax:

```
if expr:
    statement(s)
else:
    statement(s)
```

If the **expr** is true, the blocks of statements inside **if**, will be executed, and the statements inside **else** will be skipped. If **expr** is false, then the first block is skipped, and the second one is executed. Either way, execution then resumes after the second block, means **else**. Both blocks are defined by indentation, as described above. Now let's re-write the above program (Program 6.1) with if and else.

Problem 6.2: Write a program to find the largest of two integers using if-else statement and numbers must be taken as input from user.

```
1 #Step-1: Taking Input
2 num1 = int(input('Enter number:'))
3 num2 = int(input('Enter number:'))
4 #Step-2: Display both numbers
5 print(f'Numbers are : num1 = {num1}, num2 = {num2}')
6
7 #Step-3: A True Condition for num1 is largest
8 if num1 > num2:
9     print('Inside If')
10    print(f'{num1} is largest')
11 #Step-4: Automatically executes, if step-3 Condition become False
12 else:
13    print('Inside Else')
14 print(f'{num2} is largest')
```

Enter number:8

Enter number:9

Numbers are : num1 = 8, num2 = 9

```
Inside Else
9 is largest
```

See the solution carefully and the **else** clause at line 12, does not have any **expr** like **if**. It means else not designed to test any condition. After running of the above program with 8 (for num1) and 9 (for num2) as input, the if condition, **num1 > num2**, at line 8, evaluates to False, as we have entered largest value for num2. So, it is obvious that the expression of if False and else block will be executed, thereafter statements written after else will be execute. Earlier we have solved the same problem using if and now we have re-written with else clause, which improves the code efficiency. Let us answer one question, can we use an else clause without if? Answer is very straight a big NO; interpreter will raise a **SyntaxError**.

Now let's add another condition to the above program. Previously we have checked the largest of two numbers using if and else clause. Now suppose we have given two of the same numbers as input and let's test the program is ready to tackle this. Now re-execute the above code and consider 5 as input for both variables and analyze the code.

```
Enter number:5
Enter number:5
Numbers are : num1 = 5, num2 = 5
Inside Else
5 is largest
```

The output is '5 is largest', but which one, num1 or num2. This happened because we did not write the required logic to test the equality. Now question is that how to add the third condition in the program. We can do this by replacing the else clause with an if and add third if statement to code, see the modified code:

```
num1 = int(input('Enter number:'))
num2 = int(input('Enter number:'))
print(f'Numbers are : num1 = {num1}, num2 = {num2}')

#Condition 1: A True Condition for num1 is largest
if num1 > num2:
    print(f'{num1} is largest')
#Condition 2: A True condition for num2 is largest
if num2 > num1:
    print(f'{num2} is largest')
#Condition 3: A true condition for both numbers are equal
if num1 == num2:
    print('Both numbers are equal')
```

In the above code we have written three individual conditions based on their different outcome of the problem. Means, if we take input **num1** as the largest value, condition1 will become True and for num2 as largest and equality condition2 and 3 will be True. There is nothing wrong about the output of the code, but the main drawback of this approach is all three conditions will be checked by the interpreter. It is wasting of execution time and to improve we can use an else clause after any of the if statement, as follows:

```
1 num1 = int(input('Enter number:'))
```

```

2 num2 = int(input('Enter number:'))
3 print(f'Numbers are : num1 = {num1}, num2 = {num2}')
```

4

```

5 #Condition 1: A True Condition for num1 is largest
6 if num1 > num2:
7     print(f'{num1} is largest')
8 #If, condition 1 False, Else clause will execute automatically
9 else:
10    print(f'{num2} is largest')
11 #Condition 2: A true condition for both numbers are equal
12 if num1 == num2:
13    print('Both numbers are equal')
```

The second if statement, at line 8 is replaced by an else clause, it means if the first if statement is True, interpreter will skip the else clause. So, we have saved execution time, but, what if, the first if statement is False, then else at line 9 and if statement at line 12 also evaluated (for same number). Have a look at the output screen:

```

Enter number:5
Enter number:5
Numbers are : num1 = 5, num2 = 5
5 is largest
Both numbers are equal
```

We get two outputs instead of just 'Both numbers are equal'. This happened because we have placed the else clause after the first if. So, it makes the if statement at line 12 is an individual condition and that's why we get two outputs. To stop this one thing we can do, place the else clause after the second if, as follows:

```

num1 = int(input('Enter number:'))
num2 = int(input('Enter number:'))
print(f'Numbers are : num1 = {num1}, num2 = {num2}')
```

#Condition 1: A True Condition for num1 is largest

```

if num1 > num2:
    print(f'{num1} is largest')
```

#Condition 2: A true condition for num1 is largest

```

if num1 < num2:
    print(f'{num2} is largest')
```

#If, condition 2 False, Else clause will execute automatically

```

else:
    print('Both numbers are equal')
```

Now we haven't put any condition for equality, instead the else clause will take care the equality. The modified code still can produce two outputs, num1 is taken as largest value as input. It means if the first **if**, evaluates to True still the second **if** will be considered for testing and else clause gets executed. Means, to get only one output based on multiple conditions, we need to write the logic with multiple ifs, if program execution is not the concern. But in case of more than two conditions, using off one else clause, will not produce the expected output. To get this along with the reduction of execution time, we need to re-write the logic using **if-elif-else** (if-else ladder) statements.

6.1.3 The if-elif-else Statements

For branching execution based on several alternatives we can use one or more **elif** (short for else if) clauses. Python evaluates each expression in turn and executes the block corresponding to the first that is True. If none of the expression is True, and an else clause is specified, then its body is executed. Consider the required syntax for this:

```
>>> if expr:
...     statement(s)
... elif expr:
...     statement(s)
... elif expr:
...     statement(s)
... .. .
... .. .
... else:
...     statement(s)
```

We can place any number of **elif** (else if) clause, after if statement with an optional else clause at the end. The actual benefit is we can interlink many conditions together, and on execution of any expr as True, interpreter will skip the remaining **elif** and **else** (if provided) clause. At most, one of the code blocks specified will be executed. If no else clause is included, then you might see no output, if all the conditions are False, then none of the blocks will be executed.

Problem 6.3: Write a program to find the largest of two integers using if-elif-else statement and write the required conditions to test the equality of two input numbers. Execute your code 3 times with following pair of inputs (4, 5), (5, 4) and (5, 5) and the code shouldn't produce more than one output.

```
1 num1 = int(input('Enter number:'))
2 num2 = int(input('Enter number:'))
3 print(f'Numbers are : num1 = {num1}, num2 = {num2}')
4
5 #Condition 1: A True Condition for num1 is largest
6 if num1 > num2:
7     print('num1 is largest')
8 #Condition 2: A true condition for num2 is largest
9 elif num2 > num1:
10    print(f'num2 is largest')
11 #For equality
12 else:
13    print('Both numbers are equal')
```

Execution1	Execution2	Execution3
Enter number:4	Enter number:5	Enter number:5
Enter number:5	Enter number:4	Enter number:5
Numbers are : num1 = 4, num2 = 5	Numbers are : num1 = 5, num2 = 4	Numbers are : num1 = 5, num2 = 5
num2 is largest	num1 is largest	Both numbers are equal

The above code is executed 3 times with different pair of inputs, as follows:

- num1 = 4, num2 = 5, output 'num2 is largest'. For this pair of input, condition written at line 9 with elif clause evaluates to True and the first if expression is evaluating to False. Further, else clause did not consider for execution.
- num1 = 5, b = 4, output is 'num1 is largest'. For this pair of input, the if expression at line 6, evaluates to True and interpreter skipped the elif and else clause.
- num1 = 5, b = 5, output is 'Both numbers are equal'. Here all the conditions written with if and elif clause is evaluates to False and block associated within else get executed.

Program 6.4: Write a Python program that takes a student's numerical grade as input and assigns a corresponding letter grade based on the following criteria: a) 90 or above: 'A'; b) 80 to 89: 'B'; c) 70 to 79: 'C'; d) 60 to 69: 'D'; e) Below 60: 'F'.

```
grade = float(input("Enter student's grade: "))

if grade >= 90:
    letter_grade = 'A'
elif grade >= 80:
    letter_grade = 'B'
elif grade >= 70:
    letter_grade = 'C'
elif grade >= 60:
    letter_grade = 'D'
else:
    letter_grade = 'F'
print(f"The letter grade is: {letter_grade}")
```

In the above code, total 5 conditions are listed and out of these, 1 condition is tested using if (for grade 90 or more), 3 conditions are tested using elif statements (for grade ≥ 60 and < 90) and grade less 60 will be consider by else clause. For any given grade as input, program will produce the desired output and rest of the conditions will skip by the interpreter.

In the above examples, all the if and elif expr tested only single condition and based on this any blocks get executed. Sometimes it become necessary, to build a complex logic, where a decision is taken based on combined conditions or based alternative of multiple conditions. The next Section is highlighted about this fact.

6.1.4 Logical Operators with if-elif Statements

To test multiple conditions that must be met together or to check for alternative conditions, we can combine them in a single if or elif expression using logical (**and**, **or**) operators. This can improve code readability and maintainability by avoiding the need for separate condition checks. The need of logical operators can be summarized as follows:

- **Combines Necessary Conditions:** The **and** operator lets you check if all specified conditions are met before executing a code block. This ensures that only intended actions are triggered when all requirements are fulfilled.
- **Checks for Alternatives:** The **or** operator allows you to handle scenarios where at least one of the combined conditions needs to be true. This simplifies the logic if multiple possibilities lead to the same outcome.

- **Improved Readability:** Logical operators help write concise if-elif expressions, making the code more readable and easier to maintain compared to having separate checks for each condition.
- **Complex Decision-Making:** By combining conditions, you can create more intricate decision flows, making your programs more adaptable to various situations.

You can use logical (and, or) operators to combine multiple conditions or logical not to reverse the decision, within an if or elif statement. The logical operators with if-elif work as follows:

- **and operator:** The combined condition is only True if all individual conditions connected by **and** are True. For e.g. `if age >= 18 and has_id == True:` (checks both age and ID for access)
- **or operator:** The combined condition is True if at least one of the individual conditions connected by **or** is True. For e.g. `if is_weekend == True or is_holiday == True:` (checks for either weekend or holiday)
- **not operator:** Used to reverse the logical state of a single condition. **not** True is False, and **not** False is True.

Program 6.5: Write a Python program that takes a student's numerical grade (floating-point number between 0.0 and 100.0) as input. The program should then assign a corresponding letter grade based on the following criteria:

- A: 90.0 to 100.0 (inclusive)
- B: 80.0 to 89.9 (inclusive)
- C: 70.0 to 79.9 (inclusive)
- D: 60.0 to 69.9 (inclusive)
- F: Below 60.0 (including invalid grades)

```
grade = float(input("Enter your grade (0.0-100.0): "))
```

```
if grade >= 90 and grade <= 100:
    letter_grade = 'A'
elif grade >= 80 and grade < 90:
    letter_grade = 'B'
elif grade >= 70 and grade < 80:
    letter_grade = 'C'
elif grade >= 60 and grade < 70:
    letter_grade = 'D'
else:
    letter_grade = 'F' # Includes invalid grades (< 60)

print(f"Your letter grade is: {letter_grade}")
```

The first if statement checks if the grade is both greater than or equal to 90 (≥ 90) and less than or equal to 100 (≤ 100) using logical **and**. This ensures a valid grade range for an 'A'. Subsequent elif statements follow the same pattern, checking for grade ranges for 'B', 'C', and 'D' using **< and >** to define the upper and lower bounds. If none of the if or elif conditions are met (including invalid grades below 60), the else block assigns 'F' to the letter_grade variable. Finally, the program prints the assigned letter grade.

Problem 6.6: Write a Python program that takes a year as input and determines if it's a leap year. Developed your logic to satisfies the following rules:

- If a year is divisible by 4 but not by 100 (e.g., 2004) or
- A year divisible by 400 (e.g., 2000)

```
year = int(input("Enter a year: "))
```

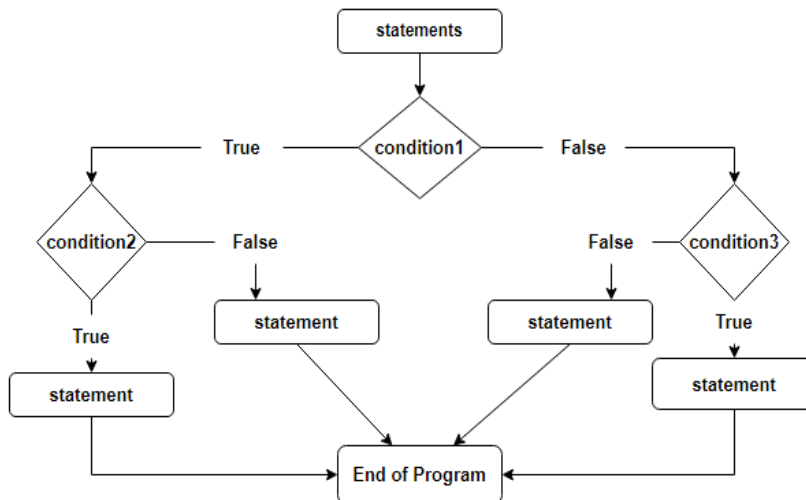
```
if (year % 4 == 0 and year % 100 != 0) or year % 400 == 0:
    print(f"{year} is a leap year!")
else:
    print(f"{year} is not a leap year.")
```

The if statement uses parentheses for proper evaluation order. The first condition (using or) checks if the year is divisible by 4 but not by 100 (`year % 4 == 0 and year % 100 != 0`). The second condition checks if the year is divisible by 400 (`year % 400 == 0`). If either condition is true, the year is a leap year. Otherwise, it's not. These examples showcase how the logical OR operator (or) can be effectively combined with if-elif statements to create more complex decision-making logic based on multiple alternative conditions.

6.1.5 Nested if-elif-else Statements

Sometimes we need to build complex logic, where multiple conditions need to be tested. Depend on True outcome program execution will follow different path, and based on False, it goes in other direction. To understand this, let's consider a flow-chart of nested if-else statement.

In the flow chart you can see based on the outcome of condition, the program execution will follow two different paths. If the condition1 become True, it will go for condition2, which will be nested inside the condition1 if or statement else it will go for condition3. In Python programming we can implement the same by nesting of an if, inside another if or inside an elif, or inside an else clause. This section highlights the working of nested if-elif-else statements based on some examples.



A nested if-else statement is an if-else statement that is nested (meaning, inside) another if, elif or else statement. With those statements we evaluate True or False conditions and make our program respond appropriately. See the syntax of nested if-else structure:

```
>>> if expr:
```

```

...     if expr:
...         statement(s)
...     else:
...         statement(s)
... elif expr:
...     if expr:
...         statement(s)
...     ... ..
...     ... ..
... else:
...     if expr:
...         statement(s)
...     else:
...         statement(s)

```

In the above syntax, we have nested if-else inside if and else block. Like this we can nest any number if-else statements or only if inside an elif clause. Let's rewrite the Leap Year Program using nested if-elif-else statements.

Problem 6.7: Write a Python program that takes a year as input and determines if it's a leap year or not. Developed your logic to satisfies the following rules:

- The given year must be divisible by 4 and then
 - If the year is divisible by 100 such as 1700 is not a leap year
 - If the year not divisible by 100 such as 2004 is a leap year
 - If the year divisible by 100 and 400 such as 2000 is a leap year
- If the year is not divisible by 4 such as 2011 is not a leap year

```

1 year = int(input('Enter a Year: '))
2
3 if year % 4 == 0:
4     if year % 100 != 0:
5         print(f'{year} is a leap year')
6     else:
7         if year % 400 == 0:
8             print(f'{year} is a leap year')
9         else:
10            print(f'{year} is not a leap year')
11 else:
12     print(f'{year} is not a leap year')

```

In the above solution, code line number (4 – 10) are nested inside the if block written at line 4. Now, these lines are part of the if block and these lines will be evaluated if the condition `year % 4 == 0`, evaluates to True. This condition will divide the given `year` by 4, if the remainder of the division operations returns zero, program control enters inside the if statement.

Next, interpreter have two situations, if the year for e.g. 1996, is not divisible by 100 and the condition at line 4, evaluates to True, that year will be displayed as leap year. Further, the condition `year % 400 == 0`, nested inside the else statement.

Problem 6.8: Write a Python program to find the largest number out of three integers. Write your logic to satisfy the equality of two and three numbers as well.

```

1 num1 = int(input('Enter a number: '))
2 num2 = int(input('Enter a number: '))
3 num3 = int(input('Enter a number: '))
4 print(f'Values are num1 = {num1}, num2 = {num2}, num3 = {num3}')
5
6 if num1 == num2 == num3:
7     print('Numbers are equal')
8 else:
9     # Find the largest number
10    if num1 > num2 and num1 > num3:
11        print(f'num1 = {num1} is largest')
12    elif num2 > num3:
13        if num2 == num1:
14            print('num1, num2 are same')
15        else:
16            print(f'num2 = {num2} is largest')
17    else:
18        if num3 == num1:
19            print('num1, num3 are same')
20        elif num3 == num2:
21            print('num3, num2 are same')
22        else:
23            print(f'num3 = {num3} is largest')

```

The above code can be broken down into two segments as follows:

- Initial Equality Check (lines 6-7): Correctly states that the first segment tests for equality using `num1 == num2 == num3`. Captures the essence of displaying "Numbers are equal" when all three numbers are the same.
- Handling Number Differences (lines 8-26): Accurately identifies the **else** clause handling cases where numbers are not equal. Clearly outlines the nested conditional structure for determining the largest number.

Breakdown of the logic within the **else** Clause, as follows:

- Checking for **num1** as Largest: Correctly identifies the condition `num1 > num2 and num1 > num3`, at line 10. Appropriately states that **num1** is declared largest when this condition holds.
- Checking for **num2**: Correctly mentions the **elif** clause at line 12, for prioritizing **num2**. Identifies the nested if-else checks for **num2**'s equality with **num1** or its designation as largest.
- Checking for **num3**: Accurately locates the final **else** clause at line 17 for handling cases where neither **num1** nor **num2** is largest. Appropriately describes the nested **if-elif-else** structure within this clause:
 - Nested **if** at line 18 checks for **num3**'s equality with **num1**.
 - Nested **elif** at line 20 checks for **num3**'s equality with **num2**.
 - Final nested **else** at line 22 declares **num3** as largest when previous conditions fail.

6.1.6 One line if statement

It is permissible to write an entire if statement in one line. The syntax is given below:

```
>>> if expr: statement
```

We can put more statements in same line, must be separated by semi-colon (;). Careful statements fall in between colon (:) and last statement must be separated by semi-colon only.

```
>>> if expr: statement_1; statement_2; statement_3
```

If the given expression after **if** is **True** then all the statements followed by colon (:) will be executed otherwise don't execute any of them. All the statements after colon (:) considered as block of **if**. See the example given below:

```
>>> if 5 > 4: print(1); print(2); print(3)
...
1
2
3
```

Note: While all of this works, and the interpreter allows it, it is generally discouraged on the grounds that it leads to poor readability, particularly for complex **if** statements.

6.1.7 Ternary Operator

Since Python 2.5, it supports one additional decision-making entity called a conditional expression (because the C language has a similar tool). In python, conditional expressions also known as conditional operator or ternary operator. See the syntax given below:

```
>>> statement1 if expr else statement2
```

Python executes **statement1**, if the conditional expression (**expr**) evaluates to **True**, or executes **statement2**, if **expr**, turns out to be **False**. We can also assign the result of conditional operator to a variable. In ternary operator the **else** part is compulsory, and if we forget to put, interpreter will raise a **SyntaxError**. Let's see an example,

```
>>> age = 18
>>> s = 'minor' if age < 21 else 'adult'
>>> print(s)
minor
```

The above code tested a person is minor or adult based on his/her age. The variable **age** is holding a value 18 and the condition **age < 21**, after **if** is evaluates to **True**. So, the **statement1** i.e. 'minor' is assigned to **s** and the **statement2** as part of **else** is ignored.

A common use of the conditional expression is to select variable assignment. For example, suppose you want to find the smallest of two numbers. Of course, there is a built-in function, **min()**, which can solve our purpose. But suppose you want to write your own code from scratch, as follows:

```
>>> a, b = 11, 12
>>> min = 0
>>> if a < b:
...     min = a
... else:
...     min = b
...
>>> print(min)
11
```

With comparison to the above code a conditional expression will be shorter and more readable, as follows:

```
>>> min = a if a < b else b
>>> print(min)
11
```

Note: The conditional expression behaves like an expression syntactically. It can be used as part of a longer expression. The conditional expression has lower precedence than virtually all the other operators, so parentheses are needed to group it by itself.

```
>>> a = b = 30
>>> c = 1+a if a < b else b+2
>>> c
32
```

Here $(1 + a)$ and $(b + 2)$ will be evaluated first then followed by the conditional expression `(31 if 30 < 30 else 32)` and 32 is assigned to the variable `c`. If we want to evaluate the conditional expression first, then we need to put the conditional expression `(a if a < b else b)` inside parenthesis, as follows:

```
c = 1 + (a if a < b else b) + 2
>>> c
33
```

The conditional expression `a<b` evaluates to **False** and the value of `b` i.e. 30 is replaced the conditional expression. Then the expression become `1 + 30 + 2` evaluates to 33 and assigned to the variable `c`.

6.2 match Statements

A match statement takes an expression and compares its value to successive patterns given as one or more case blocks. A general syntax of match statement is given below:

```
match expression:
    case match_exp1:
        pass
    case match_exp2:
        pass
    case _:
        pass
```

At first, the **expression** will be evaluated and compared with every successive case expression written as `match_exp1` to `match_exp2`. On matching the body of respective case get executed. Note that, the last case: is always written as `_` (an underscored) acts as a wildcard and gets executed, if no case matches and none of the branches is executed.

There are many situations where **match** expression become very useful. Like you want to perfume a mathematical operation of any two number given by a user and the operation he wants to perform. See the next programming example,

Problem 6.8: Write a program to take two numbers and an operator as input. Build your logic with

match statement, to execute the desired mathematical operation, based on user choice.

```
a, b = eval(input('Enter two numbers:'))
operator = input('Enter an operator from +, -, *, /:')
match operator:
    case '+':
        print(a + b)
    case '-':
        print(abs(a-b))
    case '*':
        print(a * b)
    case '/':
        print(a / b)
    case _:
        print('Wrong operator selected')
```

```
Enter two numbers:4,5
Enter an operator from +, -, *, /:+
9
Enter two numbers:6,7
Enter an operator from +, -, *, /:3
Wrong operator selected
```

In the above code sample two scenario has been considered. At first, an addition operation between 4 and 5 is performed, based on the user selected operator. Once the first case expression matched, statement(s) written as part of its body gets executes and prints 9 and all other case expressions are ignored. Secondly, we have entered a wrong operator intentionally, which is not part of the any of the case expression. Due to this the last case expression gets executed and prints 'Wrong operator selected'. We can also combine multiple literals in one case expression together using | ("or") as stated below:

```
a, b = eval(input('Enter two numbers:'))
match a+b:
    case 10|15:
        print('First Case Executed')
    case 20|25:
        print('Second Case Executed')
    case _:
        print('The Special case get Executed')
```

Two points related to match statements are highlighted in the above example. At first, any mathematical expression like 'a+b' can be used as part of match expression. Secondly, using "or" multiple case expression can be combined together, which may be helpful when, we want to execute a group of statement for different types case expression. Case expression look like unpacking assignments can be used to bind variables in a match statement:

```
point = (3,0)
match point:
    #for point = (0, 0)
    case (0, 0):
        print("Origin")
    #for point(0, 3)
    case (0, y):
        print(f"Y={y}")
```

```

#for point(3,0)
case (x, 0):
    print(f"X={x}")
#for point = (3, 3)
case (x, y):
    print(f"X={x}, Y={y}")
case _:
    raise ValueError("Not a point")

```

The first pattern (0,0) has two literals and can be thought of as an extension of the literal pattern shown above. But the next two patterns (0, y) and (x, 0) combine a literal and a variable, and the variable binds a value from the subject (point). The fourth pattern captures two values, which makes it conceptually similar to the unpacking assignment (x, y) = point.

An **if** clause can be added with a pattern, known as a “guard”. If the guard is false, match goes on to try the next case block. Note that value capturing in case patterns happens before the guard is evaluated:

```

match point:
    case point(x, y) if x == y:
        print(f"Y=X at {x}")
    case point(x, y):
        print(f"Not on the diagonal")

```

6.2.1 match vs. if-else Statements

Both **match** and **if-else** statements are used for conditional branching in Python, but they offer different approaches:

- **if-else Statements**, uses Boolean expressions to evaluate conditions and execute a specific code block based on whether the condition is True or False. It can also have chained **elif** statements for handling multiple conditions.
- **match Expressions** (since Python 3.10+), performs pattern matching on an expression's value and execute code blocks based on the matched pattern(s). It offers more flexibility and readability for complex conditional logic.
- For error handling, **if-else** might require additional checks to ensure all conditions are covered. **match** can optionally include a wildcard pattern (**_**) to handle unmatched cases for better error handling.
- For simple condition, **if-else** is sufficient and familiar. But for complex pattern matching or multiple conditions, **match** offers better readability and maintainability.

6.3 Looping Statements

This section highlights about two Python’s looping constructs statements that repeat an action over and over. At first, while loop is discussed to understand the basic mechanism of general loop constructs. The second one is for loop statement, which is used to iterate over a collection of items and executes a block of statement for each item.

While we are studying the loops, we will also cover some other statements such as **break** and **continue** commonly used with loop. Along with this we will also cover some basic built-ins such as

range, **zip** and **map** are commonly used with loop. In computer programming there are two types of iteration, known as, definite and indefinite:

- The definite iterations mean, at the start of loop the number of iterations (number of times designated block will execute) is defined.
- With indefinite iteration it is not known how many times the block of code will execute. Rather the block of code is executed repeatedly as long as the condition is **True**.

6.3.1 The while loop

Python while loop is the most basic construct of looping statement. As long as the condition at top evaluates a true value the block (normally indented) will executes repeatedly. Once the top condition become false the block of indented code will be skipped and interpreter proceed with other available statements. Now let's see the basic while loop statement.

```
>>>while expr:
```

```
    statement(s)
```

The **statement(s)** generally refers block of statements, often referred as body of the loop, should be indented just like the if statement. The control expression **expr** may contain any boolean expression evaluates to **True** or **False**. If the given **expr** evaluates to **True** then the statements as part of block or body of while will executes else loop terminates. Let's consider the following while statement:

```
>>> n = 1
>>> while n <= 5:
...     print(f'value of n is {n}')
...     n = n + 1 #updating loop variable
...
value of n is 1
value of n is 2
value of n is 3
value of n is 4
value of n is 5
```

Here is what's happening in the given code:

- The variable **n** is initialised to 1. The expr **n<=5**, at line 2, evaluates to **True**, and the body of while from line 3 – 4 executes. Inside the body the value of **n** is printed and its value gets incremented by 1, through the statement **n = n + 1**.
- The statement at line 4 is called an updation of the looping variable **n**. Once all the statements from line 3 – 4 is executed, then program execution returns to the top of the loop at line 2, and the condition **n<=5** is evaluated again. This time the condition is tested over the updated value of **n**, and it's still **True** (because value of **n** is 2 which is less than or equal to 5). So, the body of while executes again, and program control moves to top.
- This process will continue till the value of **n** become 6, which turns the expression evaluates to **False** and loop terminates. Immediately program control moves to the statements followed the loop body.

Some important points need to remember while working with while loop:

- If we forget to update the loop control variable (**n** in our case), who's value always tested before entering to the body of the loop, then loop can be executed for infinite amount of time. Consider the following while statement:

```
>>> n = 1
>>> while n <= 5:
...     print(f'value of n is {n}')
```

Here the updation statement **n = n + 1**, is removed from the body of while. So, the expression **n<=5**, always be True and due to this loop body will executes infinite amount of time.

- If we miss the initialization (**n=1**) statement then a NameError will raised, if we have used a variable in the Boolean expression, which is not declared previously.

```
>>> while n <= 5:
...     print(f'value of n is {n}')
```

Traceback (most recent call last):
File "<stdin>", line 1, in <module>
NameError: name 'n' is not defined

- It is also not necessary to update the loop control variable always incremented by 1. It can be incremented or decremented by any value, as follows:

```
>>> n = 5
>>> while n > 0:
...     print(n)
...     n = n-1
```

Here the value of **n** is initialised to 5, then at every iteration its value gets decremented by 1 (**n = n - 1**). See, the next example which prints all the even numbers from 2 to 20, where loop control variable gets updated by 2, after every iteration.

```
>>> n = 2
>>> while n <= 20:
...     print(n)
...     n = n+2
```

Now let's see another example of while loop, with list, rather than a numeric comparison:

```
>>> numbers = [1,2,3,4,5]
>>> while numbers:
...     print(numbers.pop(-1))
```

In the above code, we didn't test any expression for True or False, rather the condition at line 2, will be True, till the list become empty. Once all the items are removed using the **.pop()** function the condition become False and loop terminates. We can also put a string as part of the expression and an empty string will be evaluated to False. Details about truthy and falsy python types are already discussed in Chapter 4. You can use all of them and test with while loop as part of expression.

Problem 6.9: Write a program that asks the user for a positive integer (greater than 0). Keep prompting the user until they enter a valid positive integer.

```

status = True
while status:
    number = int(input("Enter a positive integer: "))
    if number > 0:
        status = False
    else:
        print("Invalid input. Please enter a positive integer.")
print("You entered:", number)

```

```

Enter a positive integer: -9
Invalid input. Please enter a positive integer.
Enter a positive integer: 5
You entered: 5

```

Problem 6.10: Develop a program that plays a guessing game with the user. The program thinks of a random number between 1 and 100 (inclusive). The user has to guess the number within a limited number of attempts (e.g., 7 tries). After each guess, the program provides feedback like "Too high," "Too low," or "Correct!"

```

import random

secret_number = random.randint(1, 100)
guesses_left = 7

while guesses_left > 0:
    guess = int(input("Guess a number between 1 and 100: "))
    guesses_left -= 1

    if guess == secret_number:
        print("Congratulations! You guessed the number!")
        break # Exit the loop if the guess is correct
    elif guess < secret_number:
        print("Too low. Try again.")
    else:
        print("Too high. Try again.")

if guesses_left == 0:
    print("Sorry, you ran out of guesses. The number was", secret_number)

```

```

Guess a number between 1 and 100: 70
Too high. Try again.
Guess a number between 1 and 100: 30
Too low. Try again.
Guess a number between 1 and 100: 45
Too high. Try again.
Guess a number between 1 and 100: 35
Too low. Try again.
Guess a number between 1 and 100: 38
Too low. Try again.
Guess a number between 1 and 100: 40
Too low. Try again.
Guess a number between 1 and 100: 42

```

Congratulations! You guessed the number!
 Sorry, you ran out of guesses. The number was 42

6.3.2 The for loop

A for loop is a generic iterator type in Python, used to step through the items in any ordered sequence. We can use the for loop to iterate over a string, list, dictionary, set and other built-in iterables. Let us see the general syntax of for loop:

```
>>> for var in iterable_object:
...     statement(s)
```

In the given syntax **iterable_object** is a collection of objects of same type or different type such as a list or set. The **statement(s)** inside the loop body is defined by the indentation just like if-else or while statement. The loop body will execute once for each item **var** of the **iterable_object**.

Here we can say that, every item of an **iterable_object** will be assigned to **var** at the start of the iteration. Later in the body of loop we can use the value of var, if it is necessary. Let's see an example:

```
>>> numbers = [1,2,3,4,5]
>>> for number in numbers:
...     number += 1
...     print(number)
```

Here is what's happening in the given code:

- A list **numbers** is created at line 1 with 5 items. The **for** loop header at line 2 picks the first item 1 from the list **numbers** and assigned it to **number**.
- Thereafter the body of loop from line 3 – 4 will be executed and the value of number get incremented by 1.
- After that, value of number get printed and program control backs to the loop header at line 2, to iterate over the second item of the list. This process will continue till the length of the numbers. Once all the items are iterated then program control will jump outside of the loop. This program prints the output as 2, 3, 4, 5, 6 and Outside of for loop.

In Python iterable means an object can be used in iteration. If an object is iterable, it can be passed to the built-in Python function **iter()**, which returns something called an iterator. Consider the statement given below:

```
>>> iter(numbers)
<list_iterator object at 0x000002636D2665F0>
```

In the previous example we have used a list as part of iterable object, and python supports many other iterables, such as set, tuple, dictionary, and strings etc. Some of them are provide sequential access to its items and some of them are non-sequential.

Accessing a list, set, string or a tuple are similar, means at every iteration, we are getting the items directly and for string we will get individual characters. But for dictionary, we are getting the keys only, because in dictionary items are always stored in **key:values** pair. In dictionary, to access any value we need the respective key. But for the other iterators such as string, list, tuple and set, we are directly

getting the item. Use of any non-iterable object within for loop will raise a `TypeError`. A sample code to accessing dictionary items are shown below:

```
>>> my_dict = {"name": "Alice", "age": 30, "city": "New York"}
>>> for key in my_dict:
...     print(key, ":", my_dict[key])
...
name : Alice
age : 30
city : New York
```

This for loop that iterates through the keys in the `my_dict` dictionary. The `key` variable defines a variable that can be used within the loop. This variable will store the key of each value of the dictionary item for each iteration. Further, inside the for loop, we are accessing the value associated with the key, `my_dict[key]` by passing the required key inside the square bracket (`[]`). This is not the only one way to accessing a dictionary through for loop, the detail content is discussed in Chapter 7.

When to prefer for loop over while?

Whatever the examples we have seen so far in this chapter, it represents counter-style type for loop. In comparison to while, for loop is quicker than while, so it should be our first-choice tool whenever we need to iterate through a sequence or iterable objects.

Still there are some situations when we need to iterate over second or third items in a list or to update the list. Sometimes we will see it also necessary to traverse more than one iterable object at the same time. What if we need to access all the even or odd indexed item, where we need the index. To fulfil all such requirement Python provides many built-ins for specialized iteration with for loop. These are as follows:

- **range(start, stop, step):** Generates a sequence of numbers within a specified range. Useful for creating the iteration variable in for loops.
- **enumerate(iterable, start=0):** Allows you to loop over both the items and their indices in an iterable object, discussed in Chapter 7.
- **zip(iterable1, iterable2, ...):** Combines multiple iterables element-wise, allowing you to loop through them together.
- **sorted(iterable, key=None, reverse=False):** Returns a sorted version of an iterable. You can customize the sorting behavior with the `key` and `reverse` arguments, discussed in Chapter 7.
- **filter(function, iterable):** used to create a new iterator containing elements from an existing `iterable`, for which a certain condition is true. It applies a filter based on a user-defined `function`.
- **map(function, iterable):** Applies a function to each element of an iterable and returns an iterator of the results.

6.3.3 The range() Function

The `range()` function in Python3 is just the rename of `xrange()` function of Python2. Originally, both `range()` and `xrange()` produced numbers that could be used to iterated over with for-loops. But the

former generated a list of those numbers all at once, while the latter produced numbers lazily, meaning numbers were returned one at a time as they were needed.

Now let's talk about how the **range()** function works and its totally depends how many values we have passed to it. We can tell the **range()** function, about the starting and ending value and also differences between one number and the next. There are three ways we can call the **range()** function:

- **range(stop)** takes one argument and stop informs about when to stop the iteration
- **range(start, stop)** takes two arguments and start informs about the starting value
- **range(start, stop, step)** takes three arguments, step control the differences between two values generated by **range()**.

The range(stop)

It will give range of values starts from zero (0), includes every number up to **stop**, but not including. Values generated by range object can be used as index to iterate over any object, such as string, list, set etc. Consider the statements given below:

```
>>> for i in range(5):
...     print(i, end = ' ')
...
0 1 2 3 4
```

In the output we have all the whole numbers starting from zero (0) up to but not including the **stop** (5).

The range(start, stop)

With two arguments, we are controlling the numbers range from starts to stops. It means we don't have to **start** at 0 (default start value) all the time. We can use **range()** to generate a series of numbers from predefined **start** to **stop** using **range(start, stop)**. Let's find out how to generate a range starting at 1.

```
>>> for i in range(1,5):
...     print(i, end = ' ')
...
1 2 3 4
```

In the output, we have all the whole numbers from 1 (the number is provided as the start) up to but not including 5 (the number you provided as the stop).

The range(start, stop, step)

With three arguments, we can choose not only where the series of numbers will start and stop, but also how big the difference will be between one number and the next. If you don't provide a **step**, then **range()** will automatically consider 1 as step value between two numbers.

```
>>> for i in range(1,6,2):
...     print(i, end = ' ')
...
1 3 5
```

This **range()** function produces the numbers from 1 up to 6 (not including) and difference is 2, controlled by the value of **stop**.

Note: The value of stop can be a positive or negative whole number, but not zero (0). If you try, Python interpreter raised a ValueError.

Increment with range()

To generate a sequence of numbers incrementally using the **range()** function in Python, you need to provide a positive step value as the third argument. By default, the step value is 1, which creates a sequence of numbers increasing by 1. Specifying a different step value allows you to generate sequences with larger or smaller increments. To get an idea of what this means in practice, type in the following code:

```
>>> for i in range(3,100, 25):
...     print(i, end = ' ')
...
3 28 53 78
```

Here we got a range of numbers that were each greater than the preceding number by 25, the step we have provided. Now that you've seen how you can step forwards through a range, it's time to see how you can step backwards.

Decrement with range()

If your step is positive, then you move through a series of increasing numbers and are incrementing. If your step is negative, then you move through a series of decreasing numbers and are decrementing. This allows you to go through the numbers backwards.

```
>>>for i in range(10,-5,-3):
...     print(i, end=' ')
...
10 7 4 1 -2
```

In the above example, the step value is -3. That means that you'll be decrementing by 3, the new value than its preceding value. Most importantly the start value must be greater than stop value. The most Pythonic way to create a range that decrements is to use **range(start, stop, step)**. But Python does have a built-in reversed function, if you wrap **range()** inside **reversed()**, then you can print the integers in reverse order.

```
>>>for i in reversed(range(5)):
...     print(I, end=' ')
...
4 3 2 1 0
```

range() makes it possible to iterate over a decrementing sequence of numbers, whereas **reversed()** is generally used to loop over a sequence in reverse order.

Problem 6.11: Write a program to find the sum **n** natural number start from 1 upto a given number (inclusive) **n**, as input.

```

n = int(input("Enter a positive integer: "))

if n <= 0:
    print("Invalid input. Please enter a positive integer.")
else:
    # Initialize sum to 0
    sum = 0

    # Loop from 1 to n (inclusive)
    for i in range(1, n + 1):
        print(f'{sum}+{i}=', end='')
        sum += i
        print(f'{sum}')

    print(f"The sum of natural numbers from 1 to {n} is: {sum}")

```

```

Enter a positive integer: 10
0+1=1
1+2=3
3+3=6
6+4=10
10+5=15
15+6=21
21+7=28
28+8=36
36+9=45
45+10=55
The sum of natural numbers from 1 to 10 is: 55

```

Problem 6.12: Write a program to print the multiplication table of a given number by user.

```

num = int(input("Enter a number: "))

# Loop from 1 to 10 (inclusive) to print the multiplication table
for i in range(1, 11):
    product = num * i
    print(f"{num} x {i} = {product}")

```

```

Enter a number: 5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50

```

Problem 6.13: Write a program to take input 10 integers as list and build your own logic to find the minimum and maximum number of this list.

```

1 numbers = []

```

```

2
3 # Get 10 integers as input
4 for i in range(10):
5     number = int(input("Enter an integer: "))
6     numbers.append(number)
7
8 # Initialize min and max to the first element of the list
9 min_num = numbers[0]
10 max_num = numbers[0]

11 # Iterate through the list starting from the second element
12 for number in numbers[1:]:
13     if number < min_num:
14         min_num = number
15     if number > max_num:
16         max_num = number
17
18 # Print the minimum and maximum numbers
19 print("Minimum number:", min_num)
20 print("Maximum number:", max_num)

```

This program first creates an empty list **number**, uses a loop to get 10 integers as input from the user and appends them to the list (line 4-6). Next, it initializes two variables, **min_num** and **max_num**, to the first element of the list. This is because we can't be sure if the first element is the minimum or maximum yet. The program then iterates through the list starting from the second element (since we already used the first element to initialize **min_num** and **max_num**). Inside the loop (line 12-16), it compares each element with **min_num** and **max_num**. If the element is smaller than **min_num**, it updates **min_num** with the new value. Similarly, if the element is greater than **max_num**, it updates **max_num**. Finally, after iterating through the entire list, the program prints the minimum and maximum numbers.

6.3.4 Nesting of loop

Now let's look at a for loop that's a bit more sophisticated than those we've seen so far. The next example illustrates statement nesting a loop inside another loop. Given two lists of names with some names are common in the lists. We need to create a third list containing the names present in both the lists.

```

1 list1 = ['Ram', 'Rahul', 'Jyoti', 'Shima']
2 list2 = ['Rohit', 'Ram', 'Rahul', 'Shruti', 'Jyoti']
3
4 list3 = []
5 for name1 in list1:
6     for name2 in list2:
7         if name1 == name2:
8             list3.append(name1)
9
10 print(list3)

```

The above coding example can be summarized as follows:

- The outer loop at line 5 picked a name from **list1** and compared with each name of **list2** using the condition written at line 7.
- Each names of **list2** is assigned to **name2** variable using the inner **for** loop written at line 6. Here we can see that, the inner **for** loop is indented to the outer **for** loop, means it is part of the body of outer **for** loop.
- Once the given condition **name1==name2** at line 7, evaluates to True (means **name1** of **list1** is also present in **list2**), **name1** is appended to the **list3** using **.append()** list method.
- In the above code for one iteration of outer loop, inner loop get executes 5 times which is equal to the length of **list2**. Means at one single iteration of outer loop all the names of **list2** is compared with a name of **list1**. At last, all the names common in both list are printed at line 10.

6.4 break and continue Statements

The break and continue statements are used within looping statements (such as for and while loops) to control the flow of the loop and alter its behavior. These statements serve distinct purposes in looping environment.

6.4.1 The break Statement:

Purpose: The break statement is used to terminate the execution of a loop prematurely when a certain condition is met. It allows you to exit the loop before it naturally completes all its iterations.

Usage: When a break statement is encountered within a loop, the program immediately exits the loop, and control flows to the statement immediately following the loop.

```
for i in range(1, 11):
    if i == 5:
        break # Exit the loop when i equals 5
    print(i)
```

This code will print numbers from 1 to 4 and then exit the loop when **i** is equal to 5. In this code, **range()** is used to generate sequence of numbers from 1 to 10 and break statement gets executed, once the condition with if becomes True.

Use Cases: break is commonly used when you need to exit a loop prematurely based on a specific condition, such as finding a target element in a list or stopping the loop when an error condition is met.

6.4.2 The continue Statement

Purpose: The continue statement is used to skip the current iteration of a loop and move on to the next iteration. It allows you to skip certain iterations of the loop based on a condition without exiting the loop entirely.

Usage: When a continue statement is encountered within a loop, the program skips the remaining code in the current iteration and proceeds to the next iteration.

```
for i in range(1, 12):
```

```

if i % 2 == 0:
    continue # Skip even numbers
print(i)

```

This code will print only the odd numbers from 1 to 12, on meeting the condition, at line 2, `i % 2 == 0`, `continue` statement gets executed and program control will skip all the source line from part of the for loop, will move to the next iteration of the loop.

Use Cases: `continue` is used when you want to skip certain iterations of a loop based on a condition, such as filtering or processing specific elements in an iterable.

In summary, the `break` statement is used to prematurely exit a loop, while the `continue` statement is used to skip the current iteration and move on to the next one. These statements provide control flow mechanisms that allow you to tailor the behavior of loops based on specific conditions, making your code more flexible and efficient.

6.5 The else Clause with loop

The `else` keyword with loops serves a specific purpose: it provides a way to execute a block of code after a loop has completed all its iterations without being interrupted by a `break` statement. The primary purposes of using the `else` clause with loops are as follows:

- **Checking for Completion:** You can use the `else` clause to determine whether a loop completed all its iterations without any premature termination using `break`. This can be useful for situations where you want to act or make a decision based on whether a loop ran to completion or was interrupted.
- **Error Handling:** It allows you to handle situations where a loop was expected to find something or perform some action but didn't. You can use the `else` block to raise an exception, log a message, or take other appropriate error-handling actions.

This behavior can be useful in certain situations to perform additional actions or checks after a loop has finished running. The basic programming syntax to use `else` clause at the end of any looping statement is given below:

```

for item in iterable:
    # Loop body
    if condition:
        # Condition met, break out of the loop
        break
else:
    # This block is executed if the loop completes without a break
    # Additional actions or checks can be performed here

```

The `else` clause must be written at the end of any looping construct. A loop can have any number of if-else statements nested inside it, but the outer `else` clause (written at the end of loop) doesn't have any link with if-else written as part of the loop body as indented statement. The `else` clause at the end of loop will be executed, if the condition for `break` statement didn't meet, means for complete iteration of the loop. The above claim about the working of `else` clause with loop will be better understand from the next programming example.

Problem 6.14: Take a list of 10 numbers and take input a number from user to search it. Write your logic and you are free to use any looping construct, but loop must be end with an `else` clause and print

the message “Number is not found”. Else the loop body must terminate, if the given input is present in the list and displayed “Number is present at index”.

```
numbers = [11,23,45,8,2,9,8,67,3,5]

# Get the number to search for
search_number = int(input("Enter the number to search: "))

for index, number in enumerate(numbers): # Iterate with index
    if number == search_number:
        print(f"{number} is present at index {index}")
        break # Exit the loop if found

else: # Execute only if loop completes without a break (number not found)
    print("Number is not found")
```

In this example a number is searched and for that it is compared with all the numbers present in a list object. If the **search_number** is present, then else block will not be executed. In this example, the else block is executed only if the loop completes all iterations without finding the **search_number**. It provides a clear way to communicate whether the search was successful or not.

Problem 6.15: Build a required logic to test a password is satisfying the following criteria or not. Password must be taken as input from user and maximum three attempts are allowed to form a correct password. Write the required else clause with while loop and print “Password creation successful”, if user forms a password within 3 attempts, else display “Maximum attempted reached”. The condition for password is given below:

- The length of the password minimum 8 and maximum 15.
- Password must contain one lowercase, uppercase, special case, and number from 0-9.

```
import string

# Define password criteria
min_length = 8
max_length = 15
special_chars = "!@#$%^&*()-_+={}[]|\:; '<, >. ? /"

max_attempts = 3
attempts = 0

while attempts < max_attempts:
    password = input(f"Enter your password (attempt {attempts + 1}/{max_attempts} ):")

    # Check password length
    if len(password) < min_length or len(password) > max_length:
        print(f"Password length must be between {min_length} and {max_length} characters.")
        attempts += 1
        continue # Skip to the next iteration

    # Check for lowercase, uppercase, special character, and number
    has_lowercase = any(char.islower() for char in password)
    has_uppercase = any(char.isupper() for char in password)
    has_special = any(char in special_chars for char in password)
```

```

has_number = any(char.isdigit() for char in password)

if not (has_lowercase and has_uppercase and has_special and has_number):
    print("Password must contain at least one lowercase letter, uppercase letter,
special character, and number.")
    attempts += 1
    continue # Skip to the next iteration

# Password meets all criteria
print("Password creation successful!")
break # Exit the loop if successful

else:
    print("Maximum attempts reached. Password creation failed.")

```

```

Enter your password (attempt 1/3 ): 45Ab67
Password length must be between 8 and 15 characters.
Enter your password (attempt 2/3 ): 123ABC$r12487
Password creation successful!
while loop with else clause

```

This code incorporates the following elements of python:

- imports statement to import the string module
- to match password minimum and maximum length variables `max_length` and `min_length` is used
- variable `max_attempts` is used to keep track the maximum number of attempts
- `len(password)` checks the password length falls within the allowed range
- `any()` is used to check the requirement of password characters for lowercase, uppercase, special characters, and numbers.
- if statement inside the while loop checks if all character criteria are met using logical `and` operators. If any criteria are not met, it prints an error message and increments attempts.
- The `continue` statement inside if uses to skip to the next iteration. On successful creation it prints a success message and `breaks` the loop using break.
- `else` clause executes only if the loop finishes without a break (meaning all attempts are exhausted), prints a message indicating failed password creation due to exceeded attempts.

Using the else clause with loops can make your code more expressive and help you handle situations where you want to check if a certain condition was met during the loop's execution. In summary, the else clause in loops in Python is a way to handle situations where you want to execute code after a loop has completed all its iterations without interruption, and it helps improve the readability and maintainability of your code in such scenarios.

6.6 pass statement

The `pass` keyword is a placeholder statement that does nothing when executed. It is often used as a temporary placeholder for code that you plan to write in the future or to create empty code blocks when required by Python's syntax. You can use pass within loops for various reasons, including code structure, readability, or as a placeholder for future code. Here are some common use cases for pass within loops:

- **Stubbing out loop bodies:** You can use `pass` to define an empty loop body when you're in the process of designing your code and haven't implemented the logic yet. This allows you to create the loop structure without writing the actual code at that moment.

```
>>> for item in iterable:
...     #Add your logic
...     pass
```

- **Placeholder for Future Code:** When you're working on a codebase collaboratively or planning to add specific functionality to a loop later, you can use `pass` as a placeholder to indicate that this is an intentional empty section that will be filled in later.

```
for item in iterable:
    if condition:
        # Code to handle the condition
    else:
        pass # Placeholder for future code
```

- **Maintaining Code Structure:** In some cases, you may want to maintain the structure of your code even if you don't currently need to perform any action within a loop. Using `pass` in such cases keeps the loop intact and doesn't break the code's structure.
- **Simplifying Conditional Logic:** `pass` can be used in conditional blocks inside loops to maintain consistent indentation and structure, especially when the true or false branches of the condition don't require any additional code.

Key Takeaway's

- Python uses indentation to define blocks of code. Ensure that the code inside the `if` and `else` blocks is properly indented.
- After the `if` or `else` statement, a colon (`:`) is required to indicate the start of the block.
- The condition after `if` must evaluate to a Boolean value (`True` or `False`). You can use comparison operators (`==`, `!=`, `<`, `>`, `<=`, `>=`) to form these expressions.
- Use `elif` (short for else if) to check multiple conditions in sequence.
- In Python, non-zero numbers, non-empty strings, and non-empty lists are considered `True`. Zero, empty strings/lists, and `None` are considered `False`.
- For simple conditions, you can use a one-line `if-else` statement, known as a ternary operator: `x = 'True' if condition else 'False'`.
- `for` loops are used for iterating over a sequence (like a list, tuple, dictionary, set, or string).
- Use the `range()` function to generate a sequence of numbers which can be iterated over
- Use `break` to exit the loop prematurely if a condition is met.
- Use `continue` to skip the current iteration and continue with the next one.
- An `else` block after a `for` loop will execute after the loop completes normally (i.e., not exited with a `break`).
- `while` loops continue to execute as long as a condition remains true.

Practicing Question

1. Write a program that takes the current temperature as input and gives advice on whether to wear a jacket, a t-shirt, or a sweater based on the temperature range. You can take following ranges of values, or you can go with your own conditional values:
 - If temperature less than 10 print "It's cold! Wear a jacket.
 - If temperature is in between 10 and 20 "It's cool. You should wear a sweater"

- Else, It's warm enough for a t-shirt.
- 2. Simulate a traffic light using case statements. The program should display “Stop”, “Ready”, or “Go” based on the color input (Red, Yellow, Green).
- 3. Build a simple calculator that can perform basic operations (addition, subtraction, multiplication, division) using if-else or case statements to handle different operation requests.
- 4. Write a program that calculates the price of a movie ticket based on age and time of day using nested if-else statements. Include special prices for seniors and children. Use the following list of conditions:
 - If the time is before 5 PM (matinee show), apply a discount.
 - If the time is after 5 PM (evening show), charge the regular price.
 - Within each time block, check the age of the customer:
 - If the customer is a child (age < 12), apply a child discount.
 - If the customer is a senior (age >= 60), apply a senior discount.
 - Everyone else pays the regular price.
- 5. Write a python program to use case statements to suggest activities based on the day of the week input by the user (e.g., Monday - Gym, Tuesday - Swimming).
- 6. Implement a guessing game where the user must guess a secret number. Use a while loop to allow multiple attempts and an else clause to congratulate the user if they guess correctly. Use random module to generate a random number.
- 7. Create a program to calculate the factorial of a given number using a for loop. Use an else clause to print the result after the loop completes.
- 8. Write a program that checks if a number is prime. Use a for loop to check for factors and an else clause to confirm that the number is prime if no factors are found.
- 9. Write a program that calculates the sum of all natural numbers up to a given limit using a while loop. Use an else clause to display the total sum after the loop finishes
- 10. Write a program that prints numbers from 1 to 50, but skips any number divisible by 7 using continue. If the number is greater than 35, use break to exit the loop.
- 11. Take a number as input from user and find all the prime factors of the number.

*****END OF CHAPTER*****